

Lesson 8 Sidebars

Under Pressure

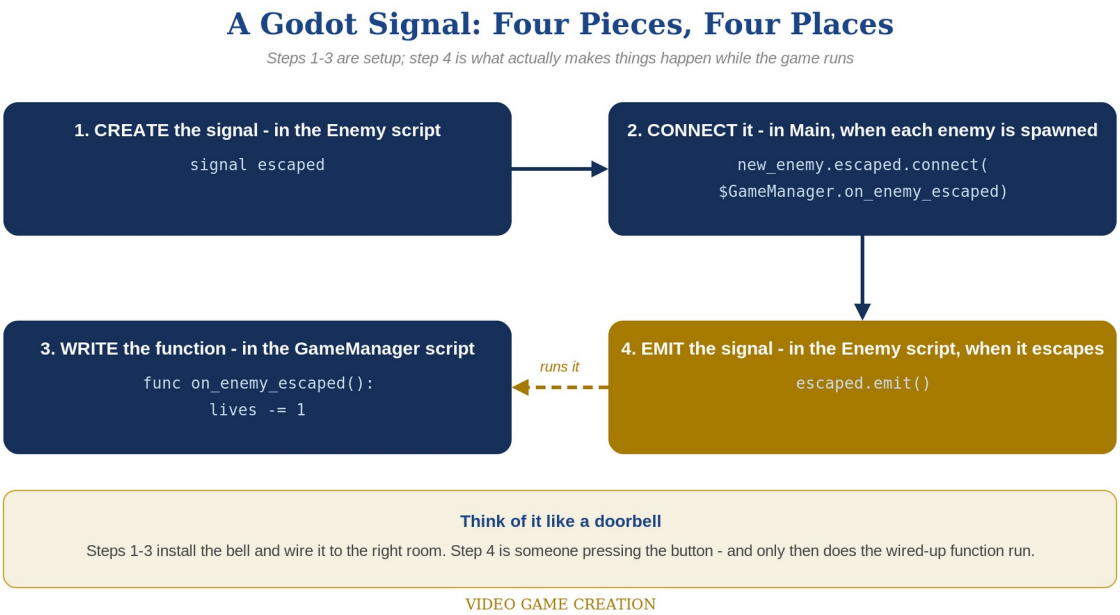
VIDEO GAME CREATION

These aren't part of the lesson's required work. Think of them the way a textbook sidebar works: a deeper, tangential look at something this lesson's build actually ran into - why one approach was chosen over the others, what the alternatives were, or an idea worth a longer look than the Handout has room for. Pull one out for a discussion moment if it fits, hand it to a curious student, or skip it entirely - nothing here is required reading.

Godot Signals: Create It, Connect It, Write the Function, Emit It

Signals confuse almost everyone the first time, and it is easy to see why: one idea is spread across three different scripts, and nothing breaks when you leave a piece out - the game just quietly does nothing. Here is the whole idea in one place.

A signal is an announcement. When something happens to an enemy - it gets destroyed, it escapes off the bottom - the enemy shouts "this just happened!" without knowing or caring who is listening. Somewhere else, a function has asked to be told. When the shout happens, Godot runs that function. Think of a doorbell: the person pressing the button does not walk into the house and do the work. They press the button, the bell is wired to a speaker in the right room, and the speaker makes the noise. That is the whole idea; the rest is just where each piece lives.



Four pieces in three scripts. Pieces 1 to 3 are set up once; piece 4 is what runs every time the event really happens.

Piece 1 - CREATE the signal (in the Enemy script)

The Enemy's script declares that this kind of announcement exists. This line does nothing by itself - it only gives the announcement a name, the way installing a doorbell button does not ring anything. Put it at the top of the script, with any other signals.

```
# enemy.tscn's script:
signal escaped
```

Piece 2 - CONNECT the signal (in Main, when each enemy is created)

Connecting is the wiring: "when THIS enemy shouts escaped, run THAT function." It happens in the Main scene's spawn code, right after each new enemy is created, because every enemy is a brand-new copy and every copy needs its own wiring. The enemy does not connect itself, and GameManager does not go looking for enemies - Main, the one place that creates them, introduces them to each other. Notice there are no parentheses after on_enemy_escaped: you are handing Godot the function itself so it can run it later, not running it right now.

```
# Main scene's script - inside the spawn loop:
# (your existing code that creates new_enemy goes here)
new_enemy.escaped.connect($GameManager.on_enemy_escaped)
```

Piece 3 - WRITE the function that will run (in the GameManager script)

This is the function the signal will run. It has to exist, with exactly that name, before the connect line can find it. This is where the actual game logic lives - the Enemy never touches lives; it only announces that it escaped, and this function decides what that means.

```
# GameManager's script:
func on_enemy_escaped():
    lives -= 1
    if lives <= 0:
        get_node("../LoseText").show()
```

Piece 4 - EMIT the signal (back in the Enemy script)

Emitting is pressing the doorbell. When the enemy actually goes past the bottom of the screen, it calls escaped.emit(). The instant that line runs, Godot looks up every function connected to escaped - here, on_enemy_escaped on GameManager - and runs it. Everything before this was preparation; this is the only line that makes something happen while the game is running.

```
# enemy.tscn's script:
func _process(delta):
    position.y += enemy_speed * delta
    if position.y > 700:
        escaped.emit()   # press the doorbell
        queue_free()
```

One escape, traced in order

- Before the game runs: the Enemy script declares signal escaped (piece 1), and GameManager has on_enemy_escaped() written (piece 3).
- When Main spawns an enemy: it connects that enemy's escaped signal to GameManager's on_enemy_escaped (piece 2).
- While the game runs: the enemy drifts down. When it passes the bottom it calls escaped.emit() (piece 4).
- Godot sees the connection from piece 2 and runs on_enemy_escaped() on GameManager, which lowers lives. Then the enemy removes itself with queue_free().

When it does nothing - and no error tells you why

- **Forgot to emit:** the signal exists and is connected, but nobody pressed the button. Lives never change. Check that escaped.emit() is really inside the branch that runs when the enemy escapes.
- **Forgot to connect:** the enemy shouts and nobody is listening. Check the spawn loop in Main - every spawned enemy needs the connect line.
- **Connected twice:** the function runs twice for one escape, so two lives disappear. Connect once, in one place.
- **Typed parentheses on the function in connect:** writing on_enemy_escaped() instead of on_enemy_escaped runs the function immediately, once, at spawn time, instead of handing it over for later.
- **Spelling mismatch:** the function name in the connect line must match the function in GameManager exactly, including capital letters; the signal name must match between piece 1, 2 and 4.

The same four pieces appear twice in this course so far. Lesson 6 used a signal named destroyed for hits; Lesson 8 added escaped for escapes. They are two separate announcements, each with its own connection and its own function, which is exactly why a hit can never accidentally cost a life.

- **A useful habit:** when a signal seems not to work, check the four pieces in order: is it created, is it connected, does the function exist, is it emitted? One of the four is almost always the missing one.

lives or __dnd_lives? Why GameMaker Has Two Answers

GameMaker has had a built-in lives variable for as long as most tutorials have existed. This lesson doesn't use it. It uses __dnd_lives - with two underscores in front. Why?

Older GameMaker tutorials, and plenty of old code, use three special global variables that GameMaker provides automatically: lives, score, and health. You could write lives = 3; anywhere without declaring it, and GameMaker kept track of it for you. Those built-ins are now deprecated, which means GameMaker still understands them for old projects but the engine's own tools no longer build on them, and they may be removed someday. The extra reason that matters most for this course: they were global, so every object in the game shared the one single value.

GameMaker's drag-and-drop blocks replaced them with something different: an instance variable. Set Lives, If Lives, and Draw Instance Lives all read and write a variable named __dnd_lives (and the Score blocks use __dnd_score, the Health blocks __dnd_health). The "dnd" stands for drag-and-drop. The two leading underscores are a naming habit programmers use to mean "this is internal plumbing - set up by the system, not a name you chose." It is a real variable with exactly that name, and the double underscore is part of the name: _dnd_lives with one underscore is a different variable that GameMaker will never create for you, and nothing will warn you if you mistype it.

Instance variable, not global

Because __dnd_lives belongs to an instance, there is one copy per object that has it. That is why this course puts it on obj_gamemanager, the single dedicated manager object introduced in Lesson 6, and why every Lives block in the Visual path has a Target drop-down that must be set to obj_gamemanager. Left at its default, a Set Lives block changes the lives of whichever object the block is sitting in - here the enemy - and the player's lives never move. That one setting is the single most likely thing to go wrong in this lesson's Visual path.

Why the code path uses the same variable

It would have been possible to leave the typed code on its own made-up variable and let the blocks use theirs. That would work, but the two paths would then be unable to share a project: a student who typed lives code and later added a Draw Instance Lives block would see the block display a number that never changes, because it is reading a different variable. Using __dnd_lives in the code means typed code, drag-and-drop blocks, and a mix of both all read and write the very same number. From another object, the code reaches it as obj_gamemanager.__dnd_lives; from inside obj_gamemanager's own events, plain __dnd_lives is enough.

- **The practical rule:** any time a lesson says lives in GameMaker, it means __dnd_lives on obj_gamemanager. The same pattern repeats with score in Lesson 9 (__dnd_score).
- **Why not invent a clearer name:** lives_remaining would read nicer, but it would be invisible to the Lives blocks. Matching the engine's own name buys a Draw Instance Lives sprite row, If Lives, and Set Lives for free.
- **A good habit to take from this:** when a tutorial's code stops working or an engine marks a feature deprecated, the fix is rarely just a rename. Ask what replaced it and what changed about how it works - here, global became per-instance - since that difference is what actually changes your design.